

Phase6 ConfusionBuffer Anki

Phase 6 — Confusion Buffer & Anki Pack (Generative & Agentic AI — scientific / chemistry agents)

Companion to the Phase-6 daily schedule (Weeks 58-63). Reinforces your critic-agent doc-review project. (Expanded edition.)

How to use: as prior packs. **Triangulation targets:** LoRA mechanics (rank/alpha), RLHF vs DPO, the evaluator-optimizer (critic) loop, ReAct/tool-calling, and how to evaluate an agent/critic.

Ranked hard-topics map

1. **Adaptation** — LoRA/QLoRA internals + when to fine-tune vs RAG vs prompt.
2. **Alignment** — RLHF pipeline, DPO, reward hacking.
3. **Agents** — ReAct, tool-calling, planning/memory/reflection, when NOT to.
4. **MCP** — exposing tools/resources; permissions.
5. **Evaluation** — agents and critics, honestly.

Anki deck (Q → A)

Deck A • Adaptation

- **Q:** LoRA — mechanism? → **A:** Freeze base weights; learn a low-rank update $\Delta W = BA$ (rank $r \ll d$) added to chosen layers → few trainable params; can be merged so there's no inference latency.
- **Q:** LoRA rank r and alpha — what do they control? → **A:** r = capacity of the update (bigger = more expressive, more params); α = scaling (effective $\Delta W = (\alpha/r) \cdot BA$) — tune together; too-small r underfits, too-large loses the efficiency point.
- **Q:** QLoRA? → **A:** Quantize the frozen base to 4-bit (NF4) + train LoRA adapters in higher precision → fine-tune large models on a single GPU.
- **Q:** Full fine-tune vs PEFT — trade-off? → **A:** Full = max capacity but expensive + risks catastrophic forgetting + a full copy per task; PEFT (LoRA/adapters) = cheap, modular, composable, minimal forgetting.
- **Q:** Catastrophic forgetting — what + mitigation? → **A:** Fine-tuning on a new task degrades old capabilities; mitigate with PEFT, replay/mixing old data, or lower LR / fewer steps.
- **Q:** Fine-tune vs prompt vs RAG — when each? → **A:** RAG for knowledge/freshness/citations; prompting for behavior needing no training; fine-tuning for a consistent skill/format/style not reachable by prompting. Often combine.
- **Q:** Continued pretraining vs instruction tuning? → **A:** Continued pretraining = more self-supervised data to shift domain knowledge; instruction tuning = SFT on (instruction, response) pairs to make it follow tasks.

Deck B • Alignment

- **Q:** RLHF pipeline? → **A:** SFT → train a reward model on human preference pairs → optimize the policy against the reward with PPO (+ a KL penalty to stay near SFT).
- **Q:** Why the KL penalty in RLHF? → **A:** Keeps the policy close to the SFT model → prevents reward hacking / gibberish that games the reward model while drifting off-distribution.
- **Q:** DPO — how does it differ? → **A:** Directly optimizes the policy on preference pairs via a classification-style loss derived from the RLHF objective — no separate reward model or RL loop.
- **Q:** RLHF vs DPO trade-off? → **A:** RLHF is flexible (online exploration, custom rewards) but complex/unstable; DPO is simpler/stable but offline and less flexible.
- **Q:** What is reward hacking (example)? → **A:** The policy maximizes the proxy reward without the intended behavior (e.g., verbose/sycophantic answers the reward model prefers) → the KL penalty + better reward data reduce it.
- **Q:** RLAI / Constitutional AI? → **A:** Use AI feedback (guided by written principles) instead of/alongside human labels to scale preference data and reduce harmful outputs.
- **Q:** What makes preference data good? → **A:** Clear, consistent comparisons on representative prompts; diverse annotators; calibrated to the behavior you actually want (not just fluency).

Deck C • Multimodal / VLM

- **Q:** CLIP — idea? → **A:** Contrastively align image and text encoders so matching pairs have high cosine similarity → zero-shot classification/retrieval via text prompts.
- **Q:** LLaVA — how is a VLM wired? → **A:** A vision encoder (e.g., CLIP-ViT) → a projection/adaptor → tokens fed into an LLM; visual-instruction-tune the whole thing.
- **Q:** Contrastive vs generative VLM? → **A:** Contrastive (CLIP) learns aligned embeddings for retrieval/zero-shot; generative (LLaVA/Flamingo) produces text conditioned on images (captioning/QA/chat).
- **Q:** How are images "tokenized" for an LLM? → **A:** Patchify → encoder features → projected to the LLM's token space (a fixed number of visual tokens per image).
- **Q:** VLM failure modes in science? → **A:** Hallucinated fine details, weak spatial/quantitative reasoning, and poor generalization to OOD imaging modalities (medical/microscopy).

Deck D • Agents + MCP

- **Q:** What makes something an "agent" (vs a single call)? → **A:** An LLM in a loop with tools + memory that plans, acts, observes, and iterates toward a goal.
- **Q:** What is ReAct? → **A:** Interleave Reasoning (a "thought") and Acting (a tool call) step by step, using observations to guide the next step → grounded multi-step behavior.
- **Q:** Tool/function calling — how does the model use a tool reliably? → **A:** Give a typed schema (name, args, description); the model emits

structured args; you validate + execute + return the result → the model continues.

- **Q:** Plan-and-execute vs ReAct? → **A:** Plan-and-execute makes a full plan up front then runs it (efficient, less adaptive); ReAct decides step-by-step (adaptive, more calls). Hybrids re-plan on failure.
- **Q:** Agent memory — short vs long term? → **A:** Short = the context window / scratchpad; long = external store (often a vector DB) retrieved as needed → persistence beyond one context.
- **Q:** What is reflection/self-critique in an agent? → **A:** The agent reviews its own output/trajectory against criteria and revises — the evaluator-optimizer pattern applied to itself.
- **Q:** Name the core agent patterns (Anthropic). → **A:** Prompt-chaining, routing, parallelization, orchestrator-worker, and evaluator-optimizer (the critic loop).
- **Q:** What is MCP (Model Context Protocol)? → **A:** A standard interface exposing tools/resources/prompts to a model, so an agent can discover and call your capability (e.g., a property+UQ model) uniformly.
- **Q:** MCP security concern? → **A:** Tools can take actions/read data → scope permissions, validate inputs/outputs, and sandbox side effects (a compromised/over-broad tool is a real risk).
- **Q:** When should you NOT use an agent? → **A:** Simple one-shot tasks → a single well-prompted call is cheaper, faster, and more reliable; agents add latency, cost, and failure surface.

Deck E · Agent & critic evaluation

- **Q:** How do you evaluate an agent? → **A:** Task success rate, tool-call correctness, steps/cost, and trajectory faithfulness — on a held-out task suite, not vibes.
- **Q:** How do you evaluate a critic? → **A:** Does its judgment correlate with ground truth/humans? Measure precision/recall of catching real errors + calibration; watch for the critic sharing the generator's blind spots.
- **Q:** Tool-use failure modes + fixes? → **A:** Wrong tool, malformed args, hallucinated tool output, infinite loops → JSON schemas, argument validation, retries, guardrails, and step limits.
- **Q:** LLM-as-judge for agents — the catch? → **A:** Same biases as any LLM judge (position/verbosity/self-preference) + it may not have ground truth for multi-step tasks → pair with programmatic checks + human review.
- **Q:** What is red-teaming an agent? → **A:** Adversarially probing for unsafe/incorrect actions, prompt injection, and tool misuse → to find failure modes before deployment.

Common misconceptions & traps

- **"Agents are always better."** For simple tasks a single well-prompted call wins on cost/latency/reliability.
- **"Fine-tuning teaches the model facts."** Mostly it shapes behavior/format; use RAG for knowledge.
- **"A huge context window removes the need for RAG/agents."** Cost scales and attention dilutes; structure still helps.
- **"The critic is objective."** A same-family critic can share the generator's blind spots — validate it against ground truth.
- **"DPO is strictly better than RLHF" (or vice versa).** Different trade-offs — DPO simpler/offline, RLHF flexible/online.
- **"More tools = a better agent."** More tools = more ways to fail; give the fewest, best-scoped tools with clear schemas.

Glossary starter

LoRA (rank r / α) / QLoRA (NF4) · PEFT vs full fine-tune · catastrophic forgetting · continued pretraining / instruction tuning (SFT) · RLHF (reward model, PPO, KL penalty) · reward hacking · DPO · RLAIF / Constitutional AI · preference data · CLIP · LLaVA / Flamingo · contrastive vs generative VLM · visual tokens · agent (plan-act-observe) · ReAct · tool/function calling (schema/validation) · plan-and-execute · agent memory (short/long, vector) · reflection/self-critique · agent patterns (routing / orchestrator-worker / evaluator-optimizer) · MCP (tools/resources, permissions) · trajectory faithfulness · agent/critic evaluation · LLM-as-judge · red-teaming / prompt injection.

Drills

Whiteboard: LoRA mechanics (rank/ α) + when to use; RLHF vs DPO + why the KL penalty; a ReAct loop; design an agent with a critic loop + how you'd evaluate the critic; MCP tool-exposure + permissions. **Blank-file:** a LoRA layer; a routing + ReAct + critic-loop agent over public literature (with tool schemas + validation); wrap a property+UQ model as an MCP tool; an eval harness for the agent (success rate, tool correctness, trajectory).

Leaving bar (cold, no notes)

LoRA/QLoRA (rank/ α) + PEFT-vs-full + when fine-tune vs RAG vs prompt; the RLHF pipeline + KL penalty + reward hacking, and DPO's trade-off; how a VLM is wired (contrastive vs generative); ReAct + tool-calling + plan-vs-react + memory/reflection; the evaluator-optimizer loop and how to evaluate the critic; MCP end-to-end + security; when an agent beats a single call (and when not).